

Agenda: Day 3

- Recap and Solution for Day 2
- What is Dynamic Analysis?
- Live Debugging with GDB
- Discussion and Resources

Recap and Solution for Day 2

- We discussed static analysis and binary patching techniques
- We talked about the two most popular static analysis tools: Ghidra and IDA
- Solution to Day 2 challenge

What is Dynamic Analysis?

Dynamic analysis is the process of reverse engineering a program during execution. Dynamic analysis generally gives us more information than Static analysis because we can see the program in action and understand how it is dealing with data in memory.

Most malware comes packed and obfuscated. Therefore, it becomes necessary to dynamically analyse the malware by executing it.

Live Debugging with GDB

<https://sourceware.org/gdb/>

<https://github.com/hugsy/gef>

Use disas main to disassemble and b main to add a breakpoint

```
gdb -q ./gdb_example

(No debugging symbols found in ./gdb_example)
gef> disas main
Dump of assembler code for function main:
0x0000000000001149 <+0>:      push    rbp
0x000000000000114a <+1>:      mov     rbp, rsp
0x000000000000114d <+4>:      sub     rsp, 0x50
0x0000000000001151 <+8>:      mov     rax, QWORD PTR fs:0x28
0x000000000000115a <+17>:     mov     QWORD PTR [rbp-0x8], rax
0x000000000000115e <+21>:     xor     eax, eax
0x0000000000001160 <+23>:     movabs  rax, 0x3335386c54585d66
0x000000000000116a <+33>:     movabs  rdx, 0x2461466450642250
0x0000000000001174 <+43>:     mov     QWORD PTR [rbp-0x20], rax
0x0000000000001178 <+47>:     mov     QWORD PTR [rbp-0x18], rdx
0x000000000000117c <+51>:     movabs  rax, 0xf6e123f66375063
0x0000000000001186 <+61>:     mov     QWORD PTR [rbp-0x10], rax
0x000000000000118a <+65>:     mov     DWORD PTR [rbp-0x44], 0x0
0x0000000000001191 <+72>:     jmp     0x11b4 <main+107>
0x0000000000001193 <+74>:     mov     eax, DWORD PTR [rbp-0x44]
0x0000000000001196 <+77>:     cdqe
0x0000000000001198 <+79>:     movzx   eax, BYTE PTR [rbp+rax*1-0x20]
0x000000000000119d <+84>:     mov     edx, eax
0x000000000000119f <+86>:     movzx   eax, BYTE PTR [rbp-0x9]
0x00000000000011a3 <+90>:     add     eax, edx
0x00000000000011a5 <+92>:     mov     edx, eax
0x00000000000011a7 <+94>:     mov     eax, DWORD PTR [rbp-0x44]
0x00000000000011aa <+97>:     cdqe
0x00000000000011ac <+99>:     mov     BYTE PTR [rbp+rax*1-0x40], dl
0x00000000000011b0 <+103>:    add     DWORD PTR [rbp-0x44], 0x1
0x00000000000011b4 <+107>:    cmp     DWORD PTR [rbp-0x44], 0x16
0x00000000000011b8 <+111>:    jle     0x1193 <main+74>
0x00000000000011ba <+113>:    lea     rax, [rip+0xe47]          # 0x2008
0x00000000000011c1 <+120>:    mov     rdi, rax
0x00000000000011c4 <+123>:    call    0x1030 <puts@plt>
0x00000000000011c9 <+128>:    lea     rax, [rip+0xe60]          # 0x2030
0x00000000000011d0 <+135>:    mov     rdi, rax
0x00000000000011d3 <+138>:    call    0x1030 <puts@plt>
0x00000000000011d8 <+143>:    mov     eax, 0x0
0x00000000000011dd <+148>:    mov     rdx, QWORD PTR [rbp-0x8]
0x00000000000011e1 <+152>:    sub     rdx, QWORD PTR fs:0x28
0x00000000000011ea <+161>:    je      0x11f1 <main+168>
0x00000000000011ec <+163>:    call    0x1040 <__stack_chk_fail@plt>
0x00000000000011f1 <+168>:    leave
0x00000000000011f2 <+169>:    ret

End of assembler dump.
gef> b main
Breakpoint 1 at 0x1151
gef>
```

Use si (step instruction) or ni (next instruction) to step through the program

The image shows a GDB terminal window with a dark background and light-colored text. At the top, the title bar reads "gdb -q ./gdb_example". Below the title bar, there is a legend: "[Legend: Modified register | Code | Heap | Stack | String]". The main content is divided into four horizontal panels. The first panel, titled "registers", displays the current state of CPU registers. The second panel, titled "stack", shows a memory dump of the stack. The third panel, titled "code:x86:64", displays the disassembled code at the current instruction pointer. The fourth panel, titled "threads", shows the current thread. The fifth panel, titled "trace", shows the current trace. The GDB prompt "gef>" is visible at the bottom left.

Use x (examine) followed by '/' and data format to examine register/memory

```
gdb -q ./gdb_example

registers
$rax : 0x0
$rbx : 0x0
$rcx : 0x0000555555557dd8 → 0x00005555555550f0 → <__do_global_ctors_aux+0000> endbr64
$rdx : 0x2461466450642250 ("PdPdFa$")
$rsp : 0x00007fffffffe160 → 0x0000000000000000
$rbp : 0x00007fffffffe1b0 → 0x00007fffffffe260 → 0x00007fffffffe2c0 → 0x0000000000000000
$rsi : 0x00007fffffffe2e8 → 0x00007fffffffe6a9 → "/home/shru/re-bootcamp-ulgc/gdb_example"
$rdi : 0x1
$rip : 0x000055555555198 → <main+004f> movzx eax, BYTE PTR [rbp+rax*1-0x20]
$r8 : 0x00007ffff7e19680 → 0x00007ffff7e1afa0 → 0x0000000000000000
$r9 : 0x00007ffff7e1afa0 → 0x0000000000000000
$r10 : 0x00007ffff7ffdf10 → 0x0000000000000000
$r11 : 0x00007ffff7ffca90 → 0x29ed34607cb60efe
$r12 : 0x00007fffffffe2e8 → 0x00007fffffffe6a9 → "/home/shru/re-bootcamp-ulgc/gdb_example"
$r13 : 0x1
$r14 : 0x00007ffff7ffd000 → 0x00007ffff7ffe3c0 → 0x0000555555554000 → 0x00010102464c457f
$r15 : 0x0000555555557dd8 → 0x00005555555550f0 → <__do_global_ctors_aux+0000> endbr64
$eflags: [zero CARRY parity ADJUST SIGN trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

stack
0x00007fffffffe160 +0x0000: 0x0000000000000000 ← $rsp
0x00007fffffffe168 +0x0008: 0x0000000000000000
0x00007fffffffe170 +0x0010: 0x0000000000000000
0x00007fffffffe178 +0x0018: 0x0000000000000000
0x00007fffffffe180 +0x0020: 0x0000000000000000
0x00007fffffffe188 +0x0028: 0x0000000000000000
0x00007fffffffe190 +0x0030: 0x3335386c54585d66
0x00007fffffffe198 +0x0038: 0x2461466450642250

code:x86:64
0x55555555191 <main+0048> jmp 0x555555551b4 <main+107>
0x55555555193 <main+004a> mov eax, DWORD PTR [rbp-0x44]
0x55555555196 <main+004d> cdqe
→ 0x55555555198 <main+004f> movzx eax, BYTE PTR [rbp+rax*1-0x20]
0x5555555519d <main+0054> mov edx, eax
0x5555555519f <main+0056> movzx eax, BYTE PTR [rbp-0x9]
0x555555551a3 <main+005a> add eax, edx
0x555555551a5 <main+005c> mov edx, eax
0x555555551a7 <main+005e> mov eax, DWORD PTR [rbp-0x44]

threads
[#0] Id 1, Name: "gdb_example", stopped 0x55555555198 in main (), reason: SINGLE STEP

trace
[#0] 0x55555555198 → main()

gef> x/b $rbp+$rax*1-0x20
0x7fffffffe190: 0x66
gef>
```

Use x/<num>i to display next instructions to be executed

```
gdb -q ./gdb_example

$rip : 0x000055555555193 → <main+004a> mov eax, DWORD PTR [rbp-0x44]
$r8 : 0x00007ffff7e19680 → 0x00007ffff7e1afa0 → 0x0000000000000000
$r9 : 0x00007ffff7e1afa0 → 0x0000000000000000
$r10 : 0x00007ffff7ffdf10 → 0x0000000000000000
$r11 : 0x00007ffff7ffca90 → 0x29ed34607cb60efe
$r12 : 0x00007ffff7ffe2e8 → 0x00007ffff7ffe6a9 → "/home/shru/re-bootcamp-ulgc/gdb_example"
$r13 : 0x1
$r14 : 0x00007ffff7ffd000 → 0x00007ffff7ffe3c0 → 0x0000555555554000 → 0x00010102464c457f
$r15 : 0x0000555555557dd8 → 0x00005555555550f0 → <__do_global_ctors_aux+0000> endbr64
$eflags: [zero CARRY parity ADJUST SIGN trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

stack
0x00007ffff7ffe160 +0x0000: 0x0000000000000000 ← $rsp
0x00007ffff7ffe168 +0x0008: 0x0000000020000000
0x00007ffff7ffe170 +0x0010: 0x000000000000006c75 ("ul"? )
0x00007ffff7ffe178 +0x0018: 0x0000000000000000
0x00007ffff7ffe180 +0x0020: 0x0000000000000000
0x00007ffff7ffe188 +0x0028: 0x0000000000000000
0x00007ffff7ffe190 +0x0030: 0x3335386c54585d66
0x00007ffff7ffe198 +0x0038: 0x2461466450642250

code:x86:64
0x55555555186 <main+003d> mov QWORD PTR [rbp-0x10], rax
0x5555555518a <main+0041> mov DWORD PTR [rbp-0x44], 0x0
0x55555555191 <main+0048> jmp 0x555555551b4 <main+107>
●→ 0x55555555193 <main+004a> mov eax, DWORD PTR [rbp-0x44]
0x55555555196 <main+004d> cdqe
0x55555555198 <main+004f> movzx eax, BYTE PTR [rbp+rax*1-0x20]
0x5555555519d <main+0054> mov edx, eax
0x5555555519f <main+0056> movzx eax, BYTE PTR [rbp-0x9]
0x555555551a3 <main+005a> add eax, edx

threads
[#0] Id 1, Name: "gdb_example", stopped 0x55555555193 in main (), reason: BREAKPOINT

trace
[#0] 0x55555555193 → main()

gef> x/10i $rip
⇒ 0x55555555193 <main+74>: mov eax,DWORD PTR [rbp-0x44]
0x55555555196 <main+77>: cdqe
0x55555555198 <main+79>: movzx eax,BYTE PTR [rbp+rax*1-0x20]
0x5555555519d <main+84>: mov edx,eax
0x5555555519f <main+86>: movzx eax,BYTE PTR [rbp-0x9]
0x555555551a3 <main+90>: add eax,edx
0x555555551a5 <main+92>: mov edx,eax
0x555555551a7 <main+94>: mov eax,DWORD PTR [rbp-0x44]
0x555555551aa <main+97>: cdqe
0x555555551ac <main+99>: mov BYTE PTR [rbp+rax*1-0x40],dl
gef>
```

See the flag appearing on the stack

```
gdb -q ./gdb_example

[ Legend: Modified register | Code | Heap | Stack | String ]

registers
$rax : 0x3
$rbx : 0x0
$rcx : 0x0000555555557dd8 → 0x0000555555550f0 → <__do_global_ctors_aux+0000> endbr64
$rdx : 0x63
$rsp : 0x00007fffffff160 → 0x0000000000000000
$rbp : 0x00007fffffff1b0 → 0x00007fffffff260 → 0x00007fffffff2c0 → 0x0000000000000000
$rsi : 0x00007fffffff2e8 → 0x00007fffffff6a9 → "/home/shru/re-bootcamp-ulg/gdb_example"
$rdi : 0x1
$rip : 0x000055555555193 → <main+004a> mov eax, DWORD PTR [rbp-0x44]
$r8 : 0x00007ffff7e19680 → 0x00007ffff7e1afa0 → 0x0000000000000000
$r9 : 0x00007ffff7e1afa0 → 0x0000000000000000
$r10 : 0x00007ffff7ffdf10 → 0x0000000000000000
$r11 : 0x00007ffff7ffca90 → 0x29ed34607cb60efe
$r12 : 0x00007fffffff2e8 → 0x00007fffffff6a9 → "/home/shru/re-bootcamp-ulg/gdb_example"
$r13 : 0x1
$r14 : 0x00007ffff7ffd000 → 0x00007ffff7ffe3c0 → 0x0000555555554000 → 0x00010102464c457f
$r15 : 0x0000555555557dd8 → 0x0000555555550f0 → <__do_global_ctors_aux+0000> endbr64
$eflags: [zero CARRY PARITY ADJUST SIGN trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

stack
0x00007fffffff160 +0x0000: 0x0000000000000000 ← $rsp
0x00007fffffff168 +0x0008: 0x0000000400000000
0x00007fffffff170 +0x0010: 0x000000006367c75 ("ulg"? )
0x00007fffffff178 +0x0018: 0x0000000000000000
0x00007fffffff180 +0x0020: 0x0000000000000000
0x00007fffffff188 +0x0028: 0x0000000000000000
0x00007fffffff190 +0x0030: 0x3335386c54585d66
0x00007fffffff198 +0x0038: 0x2461466450642250

code:x86:64
0x55555555186 <main+003d> mov QWORD PTR [rbp-0x10], rax
0x5555555518a <main+0041> mov DWORD PTR [rbp-0x44], 0x0
0x55555555191 <main+0048> jmp 0x555555551b4 <main+107>
●→ 0x55555555193 <main+004a> mov eax, DWORD PTR [rbp-0x44]
0x55555555196 <main+004d> cdqe
0x55555555198 <main+004f> movzx eax, BYTE PTR [rbp+rax*1-0x20]
0x5555555519d <main+0054> mov edx, eax
0x5555555519f <main+0056> movzx eax, BYTE PTR [rbp-0x9]
0x555555551a3 <main+005a> add eax, edx

threads
[#0] Id 1, Name: "gdb_example", stopped 0x55555555193 in main (), reason: BREAKPOINT

trace
[#0] 0x55555555193 → main()

gef>
```

Discussion and Resources

<https://www.sourceware.org/gdb/>

<https://sourceware.org/gdb/current/onlinedocs/gdb.html/>

<https://beej.us/guide/bggdb/>